FINAL PROJECT · ASSIGNMENT

Final Project Guide

Seven capstone projects, built with Claude Code

Advanced Financial Time Series · set in week 15, due in week 16

One question, one honest split, and one decision graded on data your models never saw.

One question, one model race, one graded decision

What is due, when, and with which tools

Item	Detail
Set	Teaching week 15 (Unit 16, Part 3)
Due	Week 16, together with the final exam
Team size	Individual, or a team of two
Weight	As announced in the syllabus
Tools	Claude Code, Python, the course repository
Data	The six CSVs from data/fetch_data.py; Project 4 adds public FRED series

Claude Code writes and runs the code; you are responsible for every number in the report.

Five rules apply to every project

Breaking any one caps the honest-design score at half marks

1 One question

One main asset or pair. State the question in one sentence at the top of the report.

2 Fixed split

Use the split on the next slide and write it into CLAUDE.md before any model is fitted.

3 Train only

Every parameter, threshold, lag order and setting is chosen on training data.

4 Benchmark

Beat the named naive benchmark, or say clearly that you did not.

5 Test-only grade

Score the decision on the test period alone, net of costs where you trade.

All projects share one data window

Pull with the course reference end date so every result can be checked

[Input] Command · ask Claude Code to run it

```
python data/fetch_data.py --end 2026-08-20
```

Period	Dates	Use
Training	2015-01-02 to 2021-12-31	Estimate everything
Test	2022-01-03 to 2026-08-19	Score only; never peek while choosing

Why this window

- The test period opens with the 2022 rate shock and bear market
- Walk-forward re-estimation is allowed if it uses past data only

Quote the manifest

- data_out/MANIFEST.json gives rows, dates and SHA-256
- Project 4 uses its own split and FRED data

Every forecast uses only yesterday's information

Notation used throughout the guide

$$r_t = \ln P_t - \ln P_{t-1}, \qquad \hat{y}_t = g\big(\mathcal{F}_{t-1}\big) \quad \text{(a forecast for day } t \text{ uses data up to } t-1 \text{ only)}$$

Symbols

- Returns are log returns of the adjusted close
- T is the number of training observations
- A hat marks a forecast or an estimate

The information set

- It holds everything known at the end of the previous day
- Any forecast built from later data is a leak

Claude Code builds one standard project folder

Ask for this layout in the first session

[Output] Folder Claude Code creates

```
aft-final-<your-id>/
├──── CLAUDE.md          # project rules: question, split, benchmark, no-leakage rule
├──── README.md          # one command that reproduces every result
├──── prompts.md         # log of the prompts you gave Claude Code
├──── data_out/          # created by fetch_data.py (not submitted)
├──── src/               # one script per step
├──── results/           # figures (PNG) and tables (CSV)
└──── report/            # report.md and report.pdf
```

Three files matter most

- CLAUDE.md holds the rules Claude Code must follow
- README.md rebuilds every result with one command
- prompts.md logs how you worked

Prompts 1 and 2 fix the data and the rules first

Paste in order and read each result before moving on

[Input] Prompt · 1 · Setup

```
Clone https://github.com/keerhee/advanced-financial-time-series.git, install
data/requirements.txt, and run data/fetch_data.py --end 2026-08-20.
Create the project folder aft-final-<id> with the layout in the guide.
Show me the row counts and date ranges from MANIFEST.json.
```

[Input] Prompt · 2 · Rules

```
Write CLAUDE.md for this project. Question: <one sentence>.
Training: 2015-01-02 to 2021-12-31. Test: 2022-01-03 to 2026-08-19.
Benchmark: <benchmark>. Rule: no parameter, threshold or model choice may use
test data. Every forecast for day t may use data up to day t-1 only.
Follow CLAUDE.md in every later step.
```

Replace the angle brackets with your id, question and benchmark.

Prompts 4 and 5 audit for leaks and draft the report

Prompt 3 is the starter prompt on each project's last slide

[Input] Prompt · 4 · Leakage audit

```
Audit every script in src/ for look-ahead bias. Check: scaling or
standardising with full-sample statistics, thresholds chosen on test data,
smoothed (two-sided) estimates used for trading, shift errors in signals,
and lag selection using the test period. List each issue with file and line,
fix it, re-run, and show me how the results changed.
```

[Input] Prompt · 5 · Report

```
Draft report/report.md following the report template in the guide. Use only
numbers that appear in results/. Put every figure and table in, with a
one-sentence caption that states what it shows. Then build report.pdf.
```

The audit, before and after, goes into the appendix; rewrite the conclusion in your own words.

Claude Code does the work; you own every number

Division of labour

Claude Code does	You do
Downloads data, writes and runs code	Choose the question and the design
Fixes errors and re-runs	Check that each number is plausible
Produces figures and tables	Decide what the results mean
Drafts the report	Rewrite the conclusion in your own words

You may be asked to explain any line of the report, or to re-run one step live, in a short oral check.

Choose one of seven projects

The grading scale is the same for all seven

No.	Project	Units	Level
1	Volatility forecast contest and VaR backtest	U5	★★
2	Pairs trading with a Kalman hedge ratio	U7, U9, U11	★★★
3	Regime-switching allocation with an HMM	U10	★★
4	Nowcasting US GDP with a dynamic factor model	U8, U9	★★★
5	Shock transmission from the US to Korea with a VAR	U6, U7	★★
6	Time-varying beta with the Kalman filter	U8, U9	★★★
7	Comparing MCMC samplers on stochastic volatility	U13–U15	★★★

Project 1 races three volatility models on VaR

Question: which gives the most reliable one-day 99% VaR for the S&P 500?

$$\text{EWMA:}\quad \hat{\sigma}_t^2 = \lambda\,\hat{\sigma}_{t-1}^2 + (1-\lambda)\,r_{t-1}^2, \qquad \lambda = 0.94$$

$$\text{GARCH(1,1):}\quad \sigma_t^2 = \omega + \alpha\,r_{t-1}^2 + \beta\,\sigma_{t-1}^2, \qquad \omega > 0,\ \alpha,\beta \geq 0,\ \alpha+\beta < 1$$

$$\text{GJR-}t\text{:}\quad \sigma_t^2 = \omega + \left(\alpha + \gamma\,\mathbf{1}\{r_{t-1} < 0\}\right) r_{t-1}^2 + \beta\,\sigma_{t-1}^2, \qquad \varepsilon_t \sim t_\nu \text{ (standardised)}$$

Data

- index_GSPC.csv; vix.csv as an outside reference

Benchmark

- Historical simulation: the 1% quantile of the last 250 returns

VaR is judged by breach count, clustering and loss

Evaluation on the N test days, with p = 0.01 and x breaches

$$\widehat{\mathrm{VaR}}_t = -z_p\,\hat{\sigma}_t, \quad z_{0.01} \approx -2.326, \qquad \text{breach: } r_t < -\widehat{\mathrm{VaR}}_t$$

$$LR_{uc} = -2\ln\left[(1-p)^{N-x}p^x\right] + 2\ln\left[\left(1-\tfrac{x}{N}\right)^{N-x}\left(\tfrac{x}{N}\right)^{x}\right] \sim \chi^2_1$$

$$\mathrm{QLIKE} = \frac{1}{N}\sum_t \left(\ln\hat{\sigma}_t^2 + \frac{r_t^2}{\hat{\sigma}_t^2}\right)$$

Required output

- One table: breaches, LR test, independence p-value, QLIKE
- Return plot with each VaR line; zoom on the largest VIX spike

Pitfalls

- Re-estimate on past data only, or freeze training parameters
- Never tune λ on the test period

Starter prompt for Project 1

Paste it as Prompt 3, after the setup and rules prompts

[Input] Prompt · Claude Code

```
Using index_GSPC.csv, build one-day 99% VaR forecasts for the test period
from EWMA (lambda 0.94), GARCH(1,1) and GJR-GARCH with Student-t errors
(arch package) fitted on the training period, and 250-day historical
simulation. Each forecast for day t uses data to t-1 only.
Save violation counts, Kupiec and Christoffersen tests and QLIKE to
results/var_table.csv, and the VaR plots to results/.
```

Then check

- Four methods appear in results/var_table.csv
- Each VaR for day t uses data up to t−1 only
- Breaches are compared with the expected 1%

Project 2 compares three hedge ratios for KO–PEP

Question: does a Kalman hedge ratio trade the pair better after costs?

$$\text{static: } y_t = a + \gamma\, x_t + u_t, \qquad y_t = \ln P_t^{\mathrm{KO}}, \quad x_t = \ln P_t^{\mathrm{PEP}}$$

$$\text{Kalman: } y_t = a_t + \gamma_t x_t + \varepsilon_t, \quad \begin{pmatrix} a_t \\ \gamma_t \end{pmatrix} = \begin{pmatrix} a_{t-1} \\ \gamma_{t-1} \end{pmatrix} + w_t, \quad \varepsilon_t \sim N(0, V),\ w_t \sim N(0, W)$$

Step 1: cointegration

- Engle–Granger ADF test on the training residual
- If the pair fails, report it, trade it anyway and discuss

Step 2: three hedge ratios

- Static OLS, 250-day rolling OLS and the Kalman filter
- V and W by MLE on training data; trade on the filtered estimate from t−1

An OU fit sets the rule; trading costs decide the score

Step 3 and the evaluation; benchmark is the static OLS hedge with the same rule

$$s_t = y_t - a_t - \gamma_t x_t, \qquad s_t = c + b\, s_{t-1} + e_t, \qquad \theta = -\ln b, \qquad \text{half-life} = \frac{\ln 2}{\theta}$$

$$z_t = \frac{s_t - \bar{s}}{\mathrm{sd}(s)} \ \text{(training statistics)}, \qquad R_t^{\text{net}} = R_t^{\text{gross}} - c\,|\Delta \text{position}_t|$$

Rule and score

- Enter when |z| > k, with k chosen on training data
- Exit at z = 0 or after twice the half-life
- Return, Sharpe, drawdown and trades at 0, 5, 10 and 20 bp

Pitfalls

- Smoothed Kalman estimates use future data
- Never compute the spread mean or sd on the whole sample

Starter prompt for Project 2

Paste it as Prompt 3, after the setup and rules prompts

[Input] Prompt · Claude Code

```
Using pair_KO_PEP.csv (log prices), run the Engle-Granger test on the
training period. Build three hedge ratios: static OLS, 250-day rolling OLS,
and a Kalman filter with random-walk intercept and slope (V and W by MLE on
training data, trade on the filtered estimate from t-1). Fit an AR(1) to
the training spread for the half-life, choose the entry threshold k on
training data, and backtest on the test period with 0, 5, 10, 20 bp costs.
Save the performance table and spread-with-bands plots to results/.
```

Then check

- The ADF result is printed, pass or fail
- Threshold k was chosen on training data
- The cost table covers 0, 5, 10 and 20 bp

Project 3 trades regimes found by a two-state HMM

Question: can a calm/turbulent switch beat buy-and-hold on a risk-adjusted basis?

$$P(S_t = j \mid S_{t-1} = i) = p_{ij}, \qquad r_t \mid S_t = k \sim N(\mu_k, \sigma_k^2), \qquad \text{expected duration} = \frac{1}{1 - p_{kk}}$$

Fit with hmmlearn

- GaussianHMM with two states on training returns
- At least ten random starts; keep the best likelihood
- Call the lower-volatility state calm; freeze the parameters

Report the estimates

- Annualised means and volatilities
- The transition matrix and expected durations
- Two sentences: do the regimes make economic sense?

Trade the filtered probability, never the smoothed one

Trading rule, evaluation and the required contrast; cash is the T-bill rate or zero

$$\xi_t = P(S_t = \text{calm} \mid r_1, \ldots, r_t), \qquad w_{t+1} = \mathbf{1}\{\xi_t > 0.5\}$$

Evaluation

- Benchmark: buy-and-hold of the same index
- Return, volatility, Sharpe, drawdown, switches, days invested
- Charge 5 bp per switch

Required contrast

- Rerun on smoothed probabilities or the Viterbi path
- Explain in two or three sentences why it cannot be traded
- predict_proba is smoothed: use its last row, day by day

Starter prompt for Project 3

Paste it as Prompt 3, after the setup and rules prompts

[Input] Prompt · Claude Code

```
Using index_GSPC.csv, fit a 2-state GaussianHMM (hmmlearn) on training
returns with 10 random starts, keep the best log-likelihood, and label the
lower-volatility state calm. Print the state means, volatilities,
transition matrix and expected durations. On the test period, compute the
filtered calm probability for each day as the last row of predict_proba on
data up to that day, hold the index on day t+1 when it exceeds 0.5 and cash
otherwise, with 5 bp per switch. Compare with buy-and-hold and with an
(infeasible) version using smoothed probabilities. Save the regime-shaded
chart, equity curves and performance table to results/.
```

Then check

- The calm state has the lower volatility
- Filtered probabilities are computed day by day
- The smoothed run is labelled infeasible

Project 4 nowcasts US GDP from five monthly series

The one project that uses FRED, and the one with its own split

FRED code	Series	Transform
GDPC1	Real GDP (quarterly)	×400 log growth
INDPRO	Industrial production	×100 log growth
PAYEMS	Nonfarm payrolls	×100 log growth
W875RX1	Real income less transfers	×100 log growth
CMRMTSPL	Real mfg. and trade sales	×100 log growth
RRSFS	Real retail sales	×100 log growth

Period	Use
2000-01 to 2019-12	Training: fit everything
2020-01 to 2021-12	Bridge: filtered, not scored
2022Q1 to last quarter out by 2026-08-20	Test: score only

Standardise monthly series with training-period statistics only; fetch with a new src/fetch_fred.py.

A dynamic factor model links monthly data to GDP

One factor with AR(2) dynamics; statsmodels DynamicFactorMQ implements it

$$x_{i,t} = \lambda_i f_t + e_{i,t}, \quad e_{i,t} \sim N(0, \psi_i), \qquad f_t = \phi_1 f_{t-1} + \phi_2 f_{t-2} + u_t, \quad u_t \sim N(0,1)$$

$$y_t = \lambda_y f_t + e_{y,t}, \qquad y_t^Q = \tfrac{1}{3}\left(y_t + 2y_{t-1} + 3y_{t-2} + 2y_{t-3} + y_{t-4}\right)$$

$$z_t = W_t\,(Z\alpha_t + \varepsilon_t), \qquad \alpha_t = T\alpha_{t-1} + R\,\eta_t$$

How it works

- GDP is observed in the third month of each quarter only
- W keeps the rows already published; the Kalman filter predicts through the gaps

Discuss two limitations

- Revised data, not the original vintages (ALFRED)
- The 2020–21 bridge period can distort the factor

Nowcasts are scored in pseudo real time

At each nowcast date, hide every value not yet published

Series	Approximate lag
Payrolls	About 1 week
Industrial production, retail sales	About 2–3 weeks
Income, mfg. and trade sales	About 4–6 weeks
GDP (advance estimate)	About 4 weeks after the quarter

Score

- Nowcast at month 1, 2, 3 and before release
- Benchmarks: AR(1), training mean; GDPNow as reference
- Decision: above or below the training mean?

$$\mathrm{RMSE}_h = \sqrt{\frac{1}{N}\sum_{q=1}^{N}\left(y_q^Q - \hat{y}_{q,h}^Q\right)^2}$$

Starter prompt for Project 4

Paste it as Prompt 3, after the setup and rules prompts

[Input] Prompt · Claude Code

```
Write src/fetch_fred.py to download GDPC1, INDPRO, PAYEMS, W875RX1,
CMRMTSPL, RRSFS and GDPNOW from FRED's CSV endpoint up to 2026-08-20, with a
manifest. Transform as in the guide and standardise monthly series with
2000-01 to 2019-12 statistics. Fit a one-factor DynamicFactorMQ (factor
AR(2), GDP as the quarterly series) on 2000-01 to 2019-12. For each quarter
from 2022Q1 onward, build pseudo real-time nowcasts at month 1, 2, 3 and
the month after, hiding values not yet published per the lag table. Compare
RMSE by nowcast date with an AR(1) and the training mean, add GDPNow where
available, and save the RMSE plot and tables to results/.
```

Then check

- The FRED manifest lists every series
- Unpublished values are hidden at each date
- RMSE appears for all four nowcast dates

Project 5 tests whether US moves predict Korea

Question: do yesterday's S&P 500 return and VIX change forecast today's KOSPI?

$$y_t = \left(r_t^{\mathrm{KOSPI}},\ r_{t-1}^{\mathrm{SPX}},\ \Delta\mathrm{VIX}_{t-1}\right)', \qquad y_t = c + A_1 y_{t-1} + \cdots + A_p y_{t-p} + u_t, \quad u_t \sim N(0, \Sigma)$$

Step 1: align the clocks

- Match Korean day t with the previous US trading day
- Drop days when either market is closed
- Show the first ten aligned rows in the report

Analysis

- Lag order p by AIC or BIC on training data
- Granger tests, orthogonalised IRFs and FEVD
- Justify the Cholesky order and check the reverse

Forecasts are compared with a Diebold–Mariano test

Evaluation and the trading decision on the test period

$$d_t = e^2_{\mathrm{VAR},t} - e^2_{\mathrm{AR},t}, \qquad DM = \frac{\bar{d}}{\sqrt{\widehat{\mathrm{Var}}(\bar{d})}} \sim N(0,1)$$

Score

- Benchmarks: AR(1) on the KOSPI and the training mean
- One-step RMSE and the DM test against the AR(1)
- Long/cash rule with 5 bp per trade vs buy-and-hold

Pitfall

- Misaligned dates create fake predictive power
- If a result looks too good, check the alignment first

Starter prompt for Project 5

Paste it as Prompt 3, after the setup and rules prompts

[Input] Prompt · Claude Code

```
Align kospi.csv with index_GSPC.csv and vix.csv so that Korean day t is
matched with the previous US trading day; show the first ten rows. Fit a
VAR on (KOSPI return, lagged SPX return, lagged VIX change) over the
training period with lag order by BIC. Report Granger tests, Cholesky IRFs
under two orderings, and FEVD. On the test period, compare one-step RMSE
with an AR(1), run the Diebold-Mariano test, and backtest a long/cash rule
with 5 bp costs. Save all figures and tables to results/.
```

Then check

- The first ten aligned rows look right
- Lag order was chosen on training data
- DM test and long/cash backtest both appear

Project 6 tracks AAPL's beta with a Kalman filter

Model and filter; write the loop by hand so every term appears in the code

$$y_t = \beta_t x_t + \varepsilon_t, \qquad \beta_t = \beta_{t-1} + \eta_t, \qquad \varepsilon_t \sim N(0, V),\ \eta_t \sim N(0, W)$$

$$R_t = C_{t-1} + W, \quad K_t = \frac{R_t x_t}{x_t^2 R_t + V}, \quad m_t = m_{t-1} + K_t(y_t - x_t m_{t-1}), \quad C_t = (1 - K_t x_t) R_t$$

Data

- y is the AAPL log return, x the S&P 500 log return
- returns.csv and index_GSPC.csv

Start

- m_0 from the first year's OLS beta
- $C_0 = 1$, a deliberately vague start

The likelihood picks how fast beta may move

Step 2, the band and the hedging decision; benchmarks are 60- and 250-day rolling OLS

$$\ln L(q) = -\frac{1}{2}\sum_t \left(\ln F_t + \frac{v_t^2}{F_t}\right), \quad v_t = y_t - x_t m_{t-1}, \quad F_t = x_t^2 R_t + V, \quad q = \frac{W}{V}$$

$$h_t = y_t - \hat{\beta}_{t-1} x_t \ \text{(hedged return)}, \qquad 90\%\ \text{band: } m_t \pm 1.645\sqrt{C_t}$$

Do

- Fix V at the training OLS residual variance
- Grid q from 10^{-6} to 10^{-2}; plot paths for three values

Score and discuss

- Hedged variance, its ratio to unhedged, and turnover
- Explain K as an adaptive weight; PyMC is optional

Starter prompt for Project 6

Paste it as Prompt 3, after the setup and rules prompts

[Input] Prompt · Claude Code

```
Using returns.csv (AAPL) and index_GSPC.csv, write a Kalman filter loop by
hand for y_t = beta_t x_t + e_t with a random-walk beta. Set V to the static
OLS residual variance on the training period and choose q = W/V from
{1e-6,...,1e-2} by training log-likelihood. Plot beta paths for three q
values, and the chosen path with a 90% band next to 60- and 250-day rolling
OLS betas. On the test period, hedge AAPL with the S&P 500 using the beta
from t-1 and compare hedged variance and turnover across methods. Save all
figures and tables to results/.
```

Then check

- Every Kalman term is visible in the loop
- q was chosen by training likelihood
- Hedges use the beta from t−1

Project 7 races MCMC samplers on an SV model

Three parameters plus one hidden state per day: about 1,760 unknowns

$$r_t = e^{h_t/2}\varepsilon_t, \quad h_t = \mu + \phi(h_{t-1} - \mu) + \sigma_\eta \eta_t, \quad h_1 \sim N\left(\mu, \frac{\sigma_\eta^2}{1-\phi^2}\right)$$

$$\text{priors: } \mu \sim N(0, 10), \quad \phi \sim N(0.9, 0.1^2) \text{ on } (-1, 1), \quad \sigma_\eta^2 \sim \text{IG}(2.5,\ 0.025)$$

$$\text{known-truth test: } \mu = 0, \quad \phi = 0.97, \quad \sigma_\eta = 0.2, \quad T = 1{,}000$$

Data and roles

- Demeaned S&P 500 log returns, %
- NumPy by hand; NUTS in PyMC

Rules

- Four chains each; arviz diagnostics
- Pass the known-truth test first

Gibbs samples the parameters in closed form

Sampler 1, parameter blocks; both treat h_1 as given

$$\phi \mid h, \mu, \sigma_\eta^2 \sim N(m_\phi, V_\phi) \text{ truncated to } (-1, 1)$$

$$V_\phi = \left(\frac{1}{0.1^2} + \frac{\sum_t (h_{t-1} - \mu)^2}{\sigma_\eta^2} \right)^{-1}, \qquad m_\phi = V_\phi \left(\frac{0.9}{0.1^2} + \frac{\sum_t (h_{t-1} - \mu)(h_t - \mu)}{\sigma_\eta^2} \right)$$

$$\sigma_\eta^2 \mid h, \mu, \phi \sim \text{IG}\left(2.5 + \tfrac{T-1}{2},\ 0.025 + \tfrac{1}{2} \sum_{t=2}^{T} \left(h_t - \mu - \phi(h_{t-1} - \mu) \right)^2 \right)$$

The mean is normal too

- Ask Claude Code to derive the full conditional of μ and print it in the code comments

Each hidden state gets its own Metropolis step

Sampler 1, state block: update odd days, then even days, vectorised

$$h_t \mid h_{t-1}, h_{t+1} \sim N(m_t, v), \quad m_t = \mu + \frac{\phi\,[(h_{t-1}-\mu)+(h_{t+1}-\mu)]}{1+\phi^2}, \quad v = \frac{\sigma_\eta^2}{1+\phi^2}$$

$$P(\text{accept } h_t^*) = \min\left\{1,\ \frac{N(r_t;\,0,\,e^{h_t^*})}{N(r_t;\,0,\,e^{h_t})}\right\}$$

$$\text{RW-M: } \psi^* = \psi^{(s)} + c\,z, \quad z \sim N(0, I), \qquad \text{HMC/NUTS: } H(\psi, p) = -\ln \pi(\psi) + \tfrac{1}{2} p' M^{-1} p$$

Sampler 2: joint RW Metropolis

- One proposal for all unknowns; several step sizes
- Expect failure; report it honestly

Sampler 3: NUTS in PyMC

- Centred and non-centred versions
- Compare divergences and ESS

ESS per second decides which sampler wins

Report for μ, ϕ, the vol-of-vol, and h on the day of the largest return

$$\mathrm{ESS} = \frac{S}{1 + 2\sum_{k \geq 1} \rho_k}, \qquad \text{efficiency} = \frac{\mathrm{ESS}}{\text{run time (s)}}$$

Report for each sampler

- ESS, ESS per second and R-hat (above 1.01 is not converged)
- Acceptance rates and NUTS divergences
- Trace and autocorrelation plots; overlaid posteriors

Run time

- Time a 500-draw run first
- Over 30 minutes: use 2019–2021 for every sampler
- Debug with 1,000 draws; finish with 5,000

A particle filter forecasts the test period

Fix the parameters at posterior means; 5,000 particles; benchmark is ML GARCH(1,1)

$$h_t^{(i)} = \mu + \phi\,(h_{t-1}^{(i)} - \mu) + \sigma_\eta \eta_t^{(i)} \qquad \text{1. propagate}$$

$$\mathrm{VaR}_t = -q_{0.01}\left(\tfrac{1}{N}\textstyle\sum_i N(0, e^{h_t^{(i)}})\right) \qquad \text{2. forecast before } r_t \text{ is seen}$$

$$w_t^{(i)} \propto N(r_t;\, 0,\, e^{h_t^{(i)}}) \qquad \text{3. weight and resample}$$

Score

- Count 99% VaR breaches and run the Kupiec test
- Plot filtered against smoothed volatility

Discuss

- Recommend one sampler and say why
- What does the Metropolis failure teach about many dimensions?

Starter prompt for Project 7

Paste it as Prompt 3, after the setup and rules prompts

[Input] Prompt · Claude Code

```
Fit the stochastic volatility model in the guide to demeaned S&P 500 log
returns in percent (index_GSPC.csv, training period). Write by hand in
NumPy: (1) a Gibbs sampler with conjugate updates for phi, sigma_eta^2 and
mu and a Metropolis step for each h_t, updating odd and even days in
vectorised blocks; (2) a joint random-walk Metropolis sampler with several
step sizes. Fit (3) NUTS in PyMC, centred and non-centred. Four chains each.
First check every sampler on a simulated series with mu=0, phi=0.97,
sigma_eta=0.2, T=1000. Compare ESS, ESS per second, R-hat, acceptance and
divergences with arviz. Time a 500-draw run first; if a full run would
exceed 30 minutes, use 2019-01-02 to 2021-12-31 for every sampler. Then fix
the parameters at their posterior means,
run a 5,000-particle bootstrap filter over the test period, and backtest
the 99% VaR against GARCH(1,1) with the Kupiec test. Save all tables and
plots to results/.
```

Then check

- Every sampler passed the known-truth test
- Four chains each; R-hat and ESS from arviz
- The filter forecasts before seeing r_t

Three things are submitted

Due in week 16

Repository

GitHub link or zip

The standard layout. README.md holds one command that rebuilds every figure and table from a fresh data pull.

Report (PDF)

6–10 pages + appendix

Follows the outline on the next slide. The leakage audit goes into the appendix.

prompts.md

Your prompt log

Every prompt you gave Claude Code. Not graded for style; it shows how you worked.

The report follows a fixed outline

Seven sections in this order

Section	What it contains
Question and answer	Half a page: the question, the answer and the one number behind it
Data and split	Manifest, split dates, summary statistics
Models	Equations, estimation method, training-period estimates
Test-period results	Benchmark comparison and decision performance
Robustness	At least one: another split, cost level or asset
Limitations	What could make the result wrong
Appendix	The leakage audit from Prompt 4, before and after

Honest design carries the largest single weight

100 points in seven criteria

Criterion	Points	What earns full marks
Question and design	10	One clear question; the design answers it
Data and reproducibility	15	Manifest quoted; one command rebuilds everything
Honest split and no leakage	20	Every choice made on training data; audit documented
Models	20	Equations stated; estimates correct and diagnosed
Evaluation against the benchmark	15	Proper loss or test; costs included where relevant
Interpretation and decision	15	The decision follows from the evidence; limits stated
Communication	5	Every figure has a caption that states its point

An honest loss can earn full marks; a win built on look-ahead bias cannot.

Four milestones keep the project on track

Exact dates are announced in class

Milestone	Contents	Check
M1	Data pulled, CLAUDE.md written, split fixed	Manifest and one summary plot
M2	Models fitted on the training period	Estimates and diagnostics in results/
M3	Test-period evaluation and decision	Benchmark table
M4	Leakage audit, robustness, report	report.pdf and the rebuild command

Run Prompts 1 and 2 on day one: fix the split before any model sees the data.